

# Content-Based Movie Recommendation Using TF-IDF Plot Features and Genre Encoding

Ali Kaan Demiralp   Tunay Can   Mona Hamdane  
Data Warehousing and Mining

**Abstract**—This paper presents a content-based recommendation system for movies, developed as part of a Data Mining course project. The system recommends thematically similar films using only item content features, without relying on user-item interaction data. We use the Wikipedia Movie Plots dataset (34,886 films) and apply a preprocessing pipeline that filters incomplete and uninformative records, yielding 24,959 movies. Each movie is represented by a combined feature vector composed of a TF-IDF representation of its plot text (10,000 dimensions) and a multi-hot encoding of its normalized genre labels (70 dimensions), weighted at 0.7 and 0.3 respectively after L2 normalization. Recommendations are generated using cosine similarity computed on demand. We evaluate the system over a held-out test set of 1,000 movies using Precision@5, Recall@5, and Mean Average Precision (MAP@5), with relevance defined by genre overlap. The system achieves a mean Precision@5 of 0.988, compared to 0.145 for a random baseline—a  $6.8\times$  improvement. We discuss the inherent alignment between our optimization target and evaluation criterion, and highlight cases where genre-overlap relevance underestimates semantic recommendation quality.

**Index Terms**—content-based filtering, recommendation system, TF-IDF, cosine similarity, data mining, text mining

## I. INTRODUCTION

The volume of digital media available to consumers has grown to a scale at which manual discovery is no longer feasible. A user choosing among tens of thousands of films cannot reasonably browse the full catalog, and the relevance of any individual item to that user is highly conditional on personal interests. Recommendation systems address this problem by automating the selection of items most likely to be relevant to a given user or query [1].

Two principal approaches dominate the literature. *Collaborative filtering* learns from patterns in user-item interaction data, recommending items that similar users have liked. *Content-based filtering*, by contrast, recommends items based on the features of the items themselves, comparing each candidate to items the user has previously engaged with [2]. Collaborative filtering typically achieves stronger personalization where rich interaction data exists, but suffers from the *cold-start problem*: it cannot recommend items or serve users for which interaction histories are unavailable. Content-based methods are immune to this limitation, since they operate purely on item content, making them suitable for datasets lacking explicit user feedback.

In this work, we develop a content-based recommendation system for movies. The dataset used, Wikipedia Movie Plots, contains rich item-level content (plot synopses, genre tags, production metadata) but no user-item interaction records. We

therefore implement an item-to-item recommender: given a query movie, the system returns the five most similar films based on content features. Each movie is represented by a combined feature vector that integrates a TF-IDF representation of its plot text with a multi-hot encoding of its normalized genre labels, weighted to balance textual and categorical contributions. Cosine similarity over this combined representation drives the recommendation function.

The remainder of this paper is organized as follows. Section II describes the dataset and preprocessing steps. Section III details the feature vectorization and similarity computation methodology. Section IV presents the system architecture. Section V reports experimental results and quantitative evaluation. Section VI interprets the findings and acknowledges limitations. Section VII concludes and outlines directions for future work.

## II. DATASET DESCRIPTION

The dataset used in this work is the *Wikipedia Movie Plots* dataset, publicly available on Kaggle<sup>1</sup>. It contains 34,886 movie records sourced from Wikipedia, each described by eight attributes: release year, title, origin or production country, director, principal cast, genre, source page URL, and a free-text plot synopsis. The dataset contains no user-item interaction records or rating information, which determines our choice of an item-to-item content-based recommendation approach. Table I summarizes the key properties of the dataset before and after preprocessing.

TABLE I  
DATASET SUMMARY

| Property                           | Value                          |
|------------------------------------|--------------------------------|
| Source                             | Wikipedia Movie Plots (Kaggle) |
| Total movies (raw)                 | 34,886                         |
| Total movies (after preprocessing) | 24,959                         |
| Number of users                    | None (no interaction data)     |
| Number of unique genres            | 70                             |
| Mean plot length (processed)       | 231.5 tokens                   |
| Median plot length (processed)     | 196.0 tokens                   |
| Year range                         | 1903 – 2017                    |

### A. Content Features

For each movie, the dataset provides two principal content features relevant to content-based recommendation: a free-text

<sup>1</sup><https://www.kaggle.com/datasets/jrobischon/wikipedia-movie-plots>

TABLE II  
PREPROCESSING PIPELINE RESULTS

| Stage                  | Remaining     | Dropped      | Reason                |
|------------------------|---------------|--------------|-----------------------|
| Original dataset       | 34,886        | —            | Starting point        |
| Drop unknown genres    | 28,775        | 6,111        | No genre signal       |
| Drop short plots (<50) | 26,550        | 2,225        | Insufficient text     |
| Genre frequency cutoff | 24,959        | 1,591        | Long-tail artifacts   |
| <b>Final</b>           | <b>24,959</b> | <b>9,927</b> | <b>71.5% retained</b> |

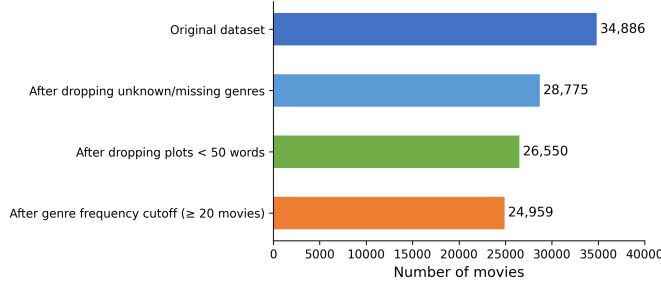


Fig. 1. Preprocessing funnel showing the number of movies retained at each filtering stage.

*plot synopsis* and a comma-separated *genre* field. Plot length varies considerably, from very short stub entries to extended multi-paragraph descriptions, with a mean of 372 words and a median of 284 words in the raw dataset. The genre field is informally tagged — the raw column contains over 2,000 distinct strings, including legitimate genres, formatting variations (e.g., *sci-fi* versus *science fiction*), and tokenization artifacts (e.g., Wikipedia citation markers and compound strings such as *action / adventure / comedy*). Director and cast fields are also present but are not used in this work, in order to preserve a focused feature space dominated by plot semantics and genre membership.

### B. Preprocessing Pipeline

The raw dataset is unsuitable for direct vectorization: records with missing or uninformative content corrupt both the feature space and the evaluation. We therefore apply a four-stage preprocessing pipeline, illustrated in Fig. 1 and summarized in Table II.

The first filter removes records where the genre field is missing, empty, or labeled *unknown*; since our evaluation strategy uses genre overlap as the relevance signal (Section V), records without genre information cannot meaningfully participate in either training or evaluation. The second filter removes records whose plot synopsis contains fewer than 50 words, the threshold below which TF-IDF vectors become too sparse to reliably encode semantic content. The third filter addresses the long tail of the genre vocabulary: after splitting the raw genre strings on multiple delimiters (commas, slashes, ampersands, hyphens flanked by whitespace) and normalizing case and whitespace, the resulting genre tokens still contain numerous tokenization artifacts appearing in only one or two records. We retain only genres that appear in at least 20 movies

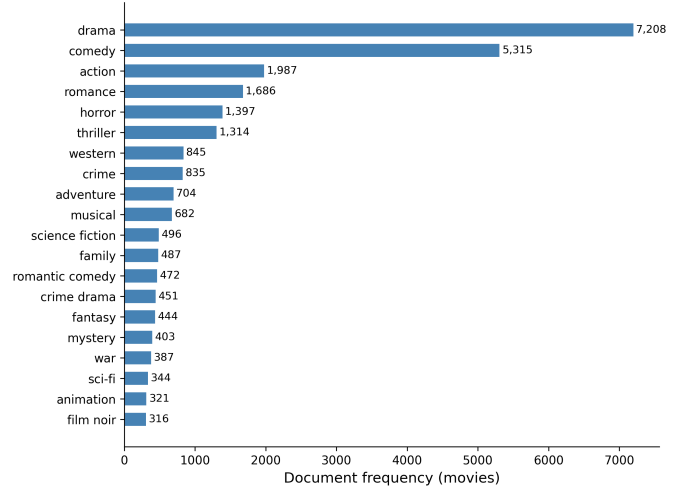


Fig. 2. Distribution of the 20 most frequent normalized genres in the cleaned dataset.

(approximately 0.075% of the dataset), reducing the genre vocabulary from over 900 raw tokens to 70 well-supported categories. Movies whose entire genre set is removed by this cutoff are dropped as well.

Following filtering, plot text is normalized through a standard text-cleaning pipeline: lowercasing, removal of punctuation and digits, English stopword removal using the NLTK stopword list, and lemmatization using the WordNet lemmatizer [3]. Figure 2 shows the resulting distribution of the 20 most frequent genres in the cleaned dataset; *drama* and *comedy* together account for approximately half of all movie-genre tags, with a long-tail distribution of more specialized categories.

## III. METHODOLOGY

This section describes the construction of the combined feature representation used for similarity computation. Each movie is represented as a single high-dimensional vector formed by concatenating two components: a TF-IDF representation of its cleaned plot text and a multi-hot encoding of its normalized genre set. The two components are independently L2-normalized and combined with empirically chosen weights.

### A. TF-IDF Plot Vectorization

Term Frequency-Inverse Document Frequency (TF-IDF) is a standard weighting scheme that scores each term in a document relative to its frequency in that document and its rarity across the corpus [4]. For a term  $t$  in document  $d$  across a corpus  $D$ , the TF-IDF score is defined as

$$\text{tfidf}(t, d, D) = \text{tf}(t, d) \cdot \log \frac{|D|}{|\{d \in D : t \in d\}|}, \quad (1)$$

where  $\text{tf}(t, d)$  denotes the frequency of  $t$  in  $d$ . We use scikit-learn's `TfidfVectorizer` [5] with sublinear term frequency scaling, replacing raw  $\text{tf}$  with  $1 + \log(\text{tf})$  to dampen the effect of highly repetitive terms.

We configure the vectorizer with the following hyperparameters: a vocabulary cap of 10,000 most frequent terms; an n-gram range of (1, 2) to capture both unigrams and bigrams; a minimum document frequency of 3, excluding terms appearing in fewer than three movies; and a maximum document frequency of 0.8, excluding terms appearing in more than 80% of movies. The lower bound removes idiosyncratic tokens that contribute noise, while the upper bound removes overly generic terms that fail to discriminate between movies. Applied to the 24,959 cleaned plots, this produces a sparse matrix of dimension  $24,959 \times 10,000$  with a density of approximately 1.45%.

### B. Multi-Hot Genre Encoding

Each movie's genre set is encoded as a binary indicator vector over the 70-dimensional genre vocabulary established during preprocessing. For a movie  $m$  with genre set  $G_m \subseteq \mathcal{G}$ , the genre vector  $\mathbf{g}_m \in \{0, 1\}^{|\mathcal{G}|}$  is defined by

$$g_{m,i} = \begin{cases} 1 & \text{if } \mathcal{G}_i \in G_m \\ 0 & \text{otherwise} \end{cases} \quad (2)$$

This encoding preserves the multi-label nature of genre membership: a movie tagged as both *crime* and *drama* produces a vector with two non-zero entries. Encoding the full dataset yields a sparse binary matrix of dimension  $24,959 \times 70$ .

### C. L2 Normalization and Weighted Combination

The two component matrices differ substantially in scale: the TF-IDF matrix has 10,000 dimensions with continuous weights, while the genre matrix has 70 binary dimensions. Naive concatenation would allow the higher-dimensional TF-IDF vector to dominate similarity computations regardless of any intended weighting. To bring the components onto a comparable scale, we L2-normalize each row of each matrix independently:

$$\hat{\mathbf{v}} = \frac{\mathbf{v}}{\|\mathbf{v}\|_2}. \quad (3)$$

After normalization, both component vectors have unit length, and the choice of weighting expresses a clean design decision rather than a side effect of dimensionality.

We then combine the normalized matrices by horizontal concatenation with scalar weights  $\alpha$  and  $\beta$ :

$$\mathbf{f}_m = [\alpha \cdot \hat{\mathbf{t}}_m \parallel \beta \cdot \hat{\mathbf{g}}_m], \quad (4)$$

where  $\hat{\mathbf{t}}_m$  is the normalized TF-IDF vector,  $\hat{\mathbf{g}}_m$  is the normalized genre vector, and  $\parallel$  denotes concatenation. We set  $\alpha = 0.7$  and  $\beta = 0.3$ , prioritizing semantic plot content while preserving a meaningful categorical signal from genre. The resulting combined matrix has dimensions  $24,959 \times 10,070$ .

### D. Cosine Similarity

We measure similarity between movies using the cosine similarity, which is well suited to high-dimensional sparse

vectors and is unaffected by vector magnitude [6]. For two combined movie vectors  $\mathbf{f}_i$  and  $\mathbf{f}_j$ :

$$\text{sim}(\mathbf{f}_i, \mathbf{f}_j) = \frac{\mathbf{f}_i \cdot \mathbf{f}_j}{\|\mathbf{f}_i\|_2 \|\mathbf{f}_j\|_2}. \quad (5)$$

Since each  $\hat{\mathbf{t}}_m$  and  $\hat{\mathbf{g}}_m$  is unit-normalized, the combined vector  $\mathbf{f}_m$  has  $\|\mathbf{f}_m\|_2 = \sqrt{\alpha^2 + \beta^2}$ , which is constant across all movies. Cosine similarity therefore reduces to a scaled dot product, computable efficiently against sparse matrices.

A practical consideration is that explicitly precomputing the full  $24,959 \times 24,959$  pairwise similarity matrix would require approximately 5 GB of dense storage, which is impractical on standard hardware. We instead compute similarity *on demand*: for a query movie  $q$ , we compute the row vector of similarities between  $\mathbf{f}_q$  and the entire combined feature matrix in a single sparse matrix-vector operation, returning the top- $k$  indices by descending similarity. This approach completes each query in under one second on commodity hardware.

## IV. SYSTEM DESIGN

The recommendation system is structured as a linear pipeline: raw input data flows through preprocessing into vectorization, vectorization into similarity computation, and similarity into ranking and output. Figure 3 illustrates the overall data flow.

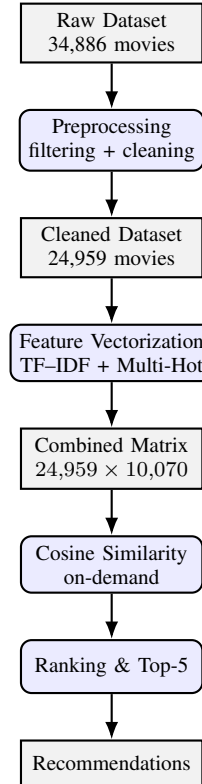


Fig. 3. Architecture of the content-based recommendation pipeline. Light gray nodes represent data states; blue nodes represent processing stages.

The pipeline begins with the raw Wikipedia Movie Plots dataset. Preprocessing applies the four-stage filter described in

Section II and normalizes plot text and genre labels, producing the cleaned dataset of 24,959 movies. Feature vectorization independently constructs the TF-IDF and multi-hot genre representations, L2-normalizes each, and concatenates them into the combined feature matrix.

At query time, the system accepts a movie identifier and computes cosine similarity between its feature vector and all other movies in the matrix. The on-demand strategy avoids materializing the full pairwise similarity matrix and keeps memory usage independent of dataset size. Self-similarity is masked, the remaining scores are sorted in descending order, and the top five candidates are returned along with their similarity scores. Already-seen items would also be filtered at this stage in a setting with user interaction history; in our item-to-item formulation, only the query movie itself is excluded.

The implementation is built on Python with NumPy, SciPy sparse matrices, scikit-learn for vectorization and similarity computation, and NLTK for text normalization. The complete pipeline executes in approximately 30 seconds end-to-end on a standard laptop, with most of the time consumed by lemmatization during preprocessing.

## V. EXPERIMENTAL RESULTS AND EVALUATION

### A. Evaluation Strategy

Since the dataset contains no user-item interaction records, conventional evaluation against historical user behavior is not possible. Following Section 9 of the project guidelines, we adopt a *category-based relevance* strategy. For a query movie  $q$  and a recommended movie  $r$ , we define  $r$  as *relevant* to  $q$  if their genre sets share at least one common element:

$$\text{rel}(q, r) = \mathbb{I}[G_q \cap G_r \neq \emptyset], \quad (6)$$

where  $G_q, G_r$  are the normalized genre sets of  $q$  and  $r$ , and  $\mathbb{I}[\cdot]$  is the indicator function. This formulation provides an objective relevance signal without requiring user feedback, although it is intentionally permissive — a property whose implications we discuss in Section VI.

We partition the 24,959 movies into a training pool (80%, 19,967 movies) and a test set (20%, 4,992 movies) using a fixed random seed. For evaluation efficiency, we randomly sample  $n = 1,000$  test queries. For each test query, the system computes cosine similarity only against the training pool and returns the top  $k = 5$  recommendations. This formulation isolates the evaluation from the trivial case where a test movie could retrieve itself or other test movies as its own neighbors.

### B. Metrics

We evaluate the system using three standard top- $k$  metrics. *Precision@5* measures the fraction of the top 5 recommendations that are relevant:

$$\text{P@5}(q) = \frac{1}{5} \sum_{i=1}^5 \text{rel}(q, r_i). \quad (7)$$

TABLE III  
EVALUATION RESULTS ( $n = 1,000, k = 5$ )

| Metric                          | Mean          | Std. Dev. |
|---------------------------------|---------------|-----------|
| Precision@5 (this system)       | 0.9880        | 0.0677    |
| Recall@5 (this system)          | 0.9880        | 0.0677    |
| MAP@5 (this system)             | 0.9799        | 0.1029    |
| Precision@5 (random baseline)   | 0.1454        | —         |
| <b>Relative lift vs. random</b> | $\times 6.79$ | —         |

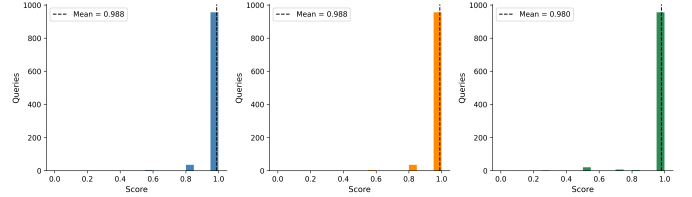


Fig. 4. Per-query distributions of Precision@5, Recall@5, and AP@5 across 1,000 sampled test queries. Dashed lines indicate the mean of each metric.

*Recall@5* measures the fraction of relevant items recovered within the top 5, using a capped denominator to account for queries with very large relevant sets:

$$\text{R@5}(q) = \frac{\sum_{i=1}^5 \text{rel}(q, r_i)}{\min(5, |R_q|)}, \quad (8)$$

where  $R_q$  is the set of all relevant items in the training pool for query  $q$ . *Average Precision@5* (AP@5) rewards systems that rank relevant items higher within the top 5:

$$\text{AP@5}(q) = \frac{1}{\min(5, |R_q|)} \sum_{i=1}^5 \text{rel}(q, r_i) \cdot \text{P@}i(q), \quad (9)$$

where  $\text{P@}i(q)$  denotes precision evaluated at cutoff  $i$ . The Mean Average Precision (MAP@5) is the mean of  $\text{AP@5}(q)$  over all test queries.

### C. Quantitative Results

Table III reports the aggregate metrics across 1,000 sampled test queries, together with a random-recommendation baseline computed by drawing 5 movies uniformly at random from the training pool and applying the same relevance criterion. Figure 4 shows the per-query distributions of each metric.

The system achieves a mean Precision@5 of 0.988, compared with 0.145 for the random baseline — a relative improvement of approximately  $6.8\times$ . Recall@5 matches Precision@5 because  $\min(5, |R_q|) = 5$  for nearly all queries: the dataset is dense enough in common genres that almost every test query has at least 5 relevant items in the training pool. MAP@5 of 0.980 is slightly lower than Precision@5, indicating that the rare misses are not uniformly distributed across rank positions; the system occasionally returns one irrelevant item among the top 5, often at lower ranks.

The score distributions in Fig. 4 are strongly concentrated at 1.0 for all three metrics: the great majority of queries achieve perfect Precision@5. A small secondary mass appears at 0.8

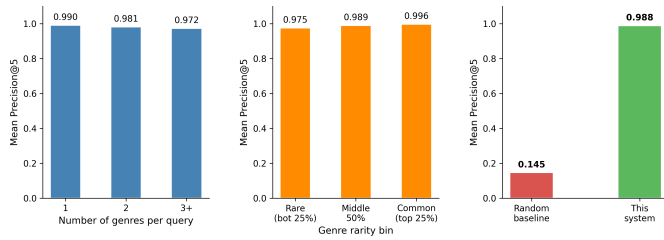


Fig. 5. Diagnostic breakdowns. (a) Mean Precision@5 by query genre count. (b) Mean Precision@5 by rarest-genre frequency bin. (c) System vs. random baseline.

(corresponding to four-out-of-five relevant), and a negligible tail extends toward zero.

#### D. Diagnostic Analysis

To better understand the conditions under which the system performs well or poorly, we conducted three diagnostic breakdowns, reported in Fig. 5. First, we stratified test queries by genre count (how many genres each query movie has). Second, we stratified queries by the document frequency of their rarest genre, binning queries into rare (bottom 25%), middle (50%), and common (top 25%) groups. Third, we computed the random baseline as described above.

The genre-count analysis shows a small inverse trend: single-genre queries achieve  $P@5 = 0.990$ , while queries with three or more genres achieve 0.972. This was contrary to our initial hypothesis that more genres should provide more opportunities for relevance matches; in practice, multi-genre queries tend to occupy hybrid niches (e.g., *horror comedy*, *romantic thriller*) whose nearest neighbors in plot space may not always preserve every component genre. The rarity analysis confirms the expected direction: queries dominated by rare genres score slightly lower (0.975) than queries dominated by common genres (0.996), reflecting the smaller pool of potential genre-matched neighbors for rare-genre queries. Both effects are small in magnitude, indicating that the system performs robustly across query difficulty.

#### E. Qualitative Examples

Table IV shows the top-5 recommendations produced by the system for two well-known query movies, illustrating the qualitative behavior of the recommender.

For *The Godfather*, the system returns four organized-crime films and a sequel from the same franchise, all with consistent thematic content. For *Star Wars*, all five recommendations belong to the same franchise, with the two original-trilogy films scoring substantially higher than the prequel and spinoff entries. The drop in similarity score from 0.54 for the original trilogy to 0.31 for the prequels reflects measurable vocabulary differences in the plot synopses despite shared characters and setting.

## VI. DISCUSSION

The system’s quantitative results are strong in absolute terms, but their interpretation requires care. This section

TABLE IV  
EXAMPLE RECOMMENDATIONS

| Rank                                                               | Recommended Movie       | Year | Sim.   |
|--------------------------------------------------------------------|-------------------------|------|--------|
| <b>Query: <i>The Godfather</i> (1972, crime drama)</b>             |                         |      |        |
| 1                                                                  | The Godfather Part III  | 1990 | 0.3433 |
| 2                                                                  | Family Business         | 1989 | 0.2973 |
| 3                                                                  | Kill the Irishman       | 2011 | 0.2735 |
| 4                                                                  | Donnie Brasco           | 1997 | 0.2632 |
| 5                                                                  | Casino                  | 1995 | 0.2606 |
| <b>Query: <i>Star Wars: A New Hope</i> (1977, science fiction)</b> |                         |      |        |
| 1                                                                  | Return of the Jedi      | 1983 | 0.5448 |
| 2                                                                  | The Empire Strikes Back | 1980 | 0.5434 |
| 3                                                                  | Revenge of the Sith     | 2005 | 0.3712 |
| 4                                                                  | Attack of the Clones    | 2002 | 0.3416 |
| 5                                                                  | The Force Unleashed     | 2013 | 0.3060 |

examines what the results indicate about the system, what they do not indicate, and the principal limitations of both the recommender and its evaluation.

#### A. Interpreting the High Evaluation Scores

A mean Precision@5 of 0.988 should not be read as evidence that the recommender is solved or that 98.8% of recommendations would satisfy a human user. Two structural factors inflate the absolute scores. First, the relevance criterion is intentionally permissive: a recommendation is counted as relevant whenever it shares any single genre with the query, and the distribution of genres in the dataset is heavy-tailed — the genre *drama* alone is present in 29% of movies and *comedy* in 21%. A random pair of movies therefore has a non-trivial probability of sharing at least one genre by chance, as confirmed by the random baseline of 0.145. Second, the same genre signal that defines relevance is also embedded in the feature representation, constituting 30% of the combined feature vector after normalization. The system explicitly optimizes for the criterion against which it is then evaluated, an alignment that is appropriate for a content-based recommender but limits what the absolute scores can demonstrate.

The meaningful comparison is therefore not against an ideal score of 1.0, but against the random baseline. The system achieves a  $6.79\times$  relative lift, demonstrating that the content-based representation captures genre-coherent similarity far more reliably than chance, and that the TF-IDF plot component contributes meaningfully beyond what the multi-hot genre encoding alone could explain.

#### B. Limitations of Genre-Overlap Relevance

The most consequential limitation of the evaluation is that genre-overlap relevance occasionally misclassifies semantically excellent recommendations as failures. The clearest case observed in our experiments was a query for *Muppet Treasure Island* (1996), tagged only as *family*: the system returned four other film adaptations of *Treasure Island* and a related adventure film as its top recommendations, all tagged as *adventure* rather than *family*. By genre overlap, every recommendation was scored as irrelevant, producing a Precision@5 of 0.0 for what is arguably an ideal set of content-based neighbors. Such

failures are rare but systematic: they occur where a query’s genre tagging is idiosyncratic relative to its natural thematic neighborhood. A more sophisticated evaluation strategy — one based, for example, on plot embedding similarity or human relevance judgments — would not penalize these cases.

### C. Limitations of the System

Several limitations of the system itself merit acknowledgment. First, the recommender uses only plot and genre content; director, cast, era, and country of origin are excluded. Including these features could potentially improve recommendations for franchises, era-specific styles, or national cinemas, at the cost of additional missing-value handling. Second, the genre vocabulary still contains near-duplicate categories (e.g., *sci-fi* and *science fiction*, *biography* and *biopic*) that fragment the genre space; a synonym-collapsing step would improve both feature quality and relevance scoring. Third, the TF-IDF representation is bag-of-words and lacks semantic generalization: it cannot recognize that *detective* and *investigator* describe similar concepts. A dense embedding approach using pre-trained language models would address this limitation but introduce significant computational and methodological complexity beyond the scope of this work. Finally, in the absence of user interaction data, the system cannot adapt to individual user preferences and produces the same recommendations for every user querying the same movie.

## VII. CONCLUSION AND FUTURE WORK

This paper presented a content-based recommendation system for movies, developed on the Wikipedia Movie Plots dataset and evaluated under a category-based relevance criterion. The system represents each movie as a combined feature vector consisting of a TF-IDF representation of its plot text and a multi-hot encoding of its genre labels, weighted at 0.7 and 0.3 respectively after L2 normalization. Cosine similarity between feature vectors drives an item-to-item recommender that returns the top five most similar movies for any query, computed on demand to avoid materializing a full pairwise similarity matrix. A four-stage preprocessing pipeline retains 24,959 of the original 34,886 movie records by filtering missing-genre rows, short plots, and long-tail tokenization artifacts.

Evaluated over 1,000 sampled test queries with relevance defined as shared genre membership, the system achieves a mean Precision@5 of 0.988, Recall@5 of 0.988, and MAP@5 of 0.980, compared with a random baseline Precision@5 of 0.145 — a  $6.79\times$  relative improvement. Diagnostic analysis confirms that the system performs robustly across query difficulty, with only small variations across genre-count and genre-rarity strata. Qualitative inspection of recommendations for well-known query movies confirms that the system surfaces thematically coherent neighbors, including franchise members, era-matched films, and topically related works across cultural origins.

Several directions remain open for future work. Incorporating additional content features — director, cast, era, country —

would likely improve recommendation diversity and personalization. Replacing the bag-of-words TF-IDF representation with a dense embedding produced by a pre-trained language model would address the system’s current inability to recognize semantic synonymy in plot text. Collapsing near-duplicate genre categories through a synonym-mapping step would sharpen the genre signal both in the feature representation and in the evaluation. Most fundamentally, the system would benefit substantially from access to user-item interaction data, enabling hybrid content-collaborative methods and evaluation strategies grounded in actual user behavior rather than categorical proxies.

## REFERENCES

- [1] F. Ricci, L. Rokach, and B. Shapira, Eds., *Recommender Systems Handbook*, 2nd ed. Springer, 2015.
- [2] P. Lops, M. de Gemmis, and G. Semeraro, “Content-based recommender systems: State of the art and trends,” in *Recommender Systems Handbook*. Springer, 2011, pp. 73–105.
- [3] S. Bird, E. Klein, and E. Loper, *Natural Language Processing with Python*. O’Reilly Media, 2009.
- [4] G. Salton and C. Buckley, “Term-weighting approaches in automatic text retrieval,” *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.
- [5] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [6] A. Singhal, “Modern information retrieval: A brief overview,” in *Bulletin of the IEEE Computer Society Technical Committee on Data Engineering*, 2001, vol. 24, no. 4, pp. 35–43.